

You are creating and assigning action items in a postmortem for an outage. The outage is over, but you need to address the root causes. You want to ensure that your team handles the action items quickly and efficiently. How should you assign owners and collaborators to action items?

- A. Assign one owner for each action item and any necessary collaborators. **Most Voted**
 - B. Assign multiple owners for each item to guarantee that the team addresses items quickly.
 - C. Assign collaborators but no individual owners to the items to keep the postmortem blameless.
 - D. Assign the team lead as the owner for all action items because they are in charge of the SRE team.
-

Your development team has created a new version of their service's API. You need to deploy the new versions of the API with the least disruption to third-party developers and end users of third-party installed applications. What should you do?

- A. Introduce the new version of the API. Announce deprecation of the old version of the API. Deprecate the old version of the API. Contact remaining users of the old API. Provide best effort support to users of the old API. Turn down the old version of the API. **Most Voted**
 - B. Announce deprecation of the old version of the API. Introduce the new version of the API. Contact remaining users on the old API. Deprecate the old version of the API. Turn down the old version of the API. Provide best effort support to users of the old API.
 - C. Announce deprecation of the old version of the API. Contact remaining users on the old API. Introduce the new version of the API. Deprecate the old version of the API. Provide best effort support to users of the old API. Turn down the old version of the API.
 - D. Introduce the new version of the API. Contact remaining users of the old API. Announce deprecation of the old version of the API. Deprecate the old version of the API. Turn down the old version of the API. Provide best effort support to users of the old API.
-

You are running an application on Compute Engine and collecting logs through Stackdriver. You discover that some personally identifiable information (PII) is leaking into certain log entry fields. You want to prevent these fields from being written in new log entries as quickly as possible. What should you do?

- A. Use the filter-record-transformer Fluentd filter plugin to remove the fields from the log entries in flight. **Most Voted**
 - B. Use the fluent-plugin-record-reformer Fluentd output plugin to remove the fields from the log entries in flight.
 - C. Wait for the application developers to patch the application, and then verify that the log entries are no longer exposing PII.
 - D. Stage log entries to Cloud Storage, and then trigger a Cloud Function to remove the fields and write the entries to Stackdriver via the Stackdriver Logging API.
-

You support a service that recently had an outage. The outage was caused by a new release that exhausted the service memory resources. You rolled back the release successfully to mitigate the impact on users. You are now in charge of the post-mortem for the outage. You want to follow Site Reliability Engineering practices when developing the post-mortem. What should you do?

- A. Focus on developing new features rather than avoiding the outages from recurring.
 - B. Focus on identifying the contributing causes of the incident rather than the individual responsible for the cause. **Most Voted**
 - C. Plan individual meetings with all the engineers involved. Determine who approved and pushed the new release to production.
 - D. Use the Git history to find the related code commit. Prevent the engineer who made that commit from working on production services.
-

You support a user-facing web application. When analyzing the application's error budget over the previous six months, you notice that the application has never consumed more than 5% of its error budget in any given time window. You hold a Service Level Objective (SLO) review with business stakeholders and confirm that the SLO is set appropriately. You want your application's SLO to more closely reflect its observed reliability. What steps can you take to further that goal while balancing velocity, reliability, and business needs? (Choose two.)

- A. Add more serving capacity to all of your application's zones.
- B. Have more frequent or potentially risky application releases.
- C. Tighten the SLO match the application's observed reliability.

- D. Implement and measure additional Service Level Indicators (SLIs) from the application. **Most Voted**
 - E. Announce planned downtime to consume more error budget, and ensure that users are not depending on a tighter SLO. **Most Voted**
-
-

You support a service with a well-defined Service Level Objective (SLO). Over the previous 6 months, your service has consistently met its SLO and customer satisfaction has been consistently high. Most of your service's operations tasks are automated and few repetitive tasks occur frequently. You want to optimize the balance between reliability and deployment velocity while following site reliability engineering best practices. What should you do? (Choose two.)

- A. Make the service's SLO more strict.
 - B. Increase the service's deployment velocity and/or risk. **Most Voted**
 - C. Shift engineering time to other services that need more reliability. **Most Voted**
 - D. Get the product team to prioritize reliability work over new features.
 - E. Change the implementation of your Service Level Indicators (SLIs) to increase coverage.
-
-

Your company follows Site Reliability Engineering principles. You are writing a postmortem for an incident, triggered by a software change that severely affected users. You want to prevent severe incident from happening in the future. What should you do?

- A. Identify engineers responsible for the incident and escalate to the senior management.
 - B. Ensure that test cases that catch errors of this type are run successfully before new software releases. **Most Voted**
 - C. Follow up with the employees who reviewed the changes and prescribe practices they should follow in the future.
 - D. Design a policy that will require on-call teams to immediately call engineers and management to discuss a plan of action if an incident occurs.
-
-

Your organization uses a change advisory board (CAB) to approve all changes to an existing service. You want to revise this process to eliminate any negative impact on the software delivery performance. What should you do? (Choose two.)

- A. Replace the CAB with a senior manager to ensure continuous oversight from development to deployment.
 - B. Let developers merge their own changes, but ensure that the team's deployment platform can roll back changes if any issues are discovered.
 - C. Move to a peer-review based process for individual changes that is enforced at code check-in time and supported by automated tests. **Most Voted**
 - D. Batch changes into larger but less frequent software releases.
 - E. Ensure that the team's development platform enables developers to get fast feedback on the impact of their changes. **Most Voted**
-

Your organization has a containerized web application that runs on-premises. As part of the migration plan to Google Cloud, you need to select a deployment strategy and platform that meets the following acceptance criteria:

1. The platform must be able to direct traffic from Android devices to an Android-specific microservice.
2. The platform must allow for arbitrary percentage-based traffic splitting
3. The deployment strategy must allow for continuous testing of multiple versions of any microservice.

What should you do?

- A. Deploy the canary release of the application to Cloud Run. Use traffic splitting to direct 10% of user traffic to the canary release based on the revision tag.
 - B. Deploy the canary release of the application to App Engine. Use traffic splitting to direct a subset of user traffic to the new version based on the IP address.
 - C. Deploy the canary release of the application to Compute Engine. Use Anthos Service Mesh with Compute Engine to direct 10% of user traffic to the canary release by configuring the virtual service.
 - D. Deploy the canary release to Google Kubernetes Engine with Anthos Service Mesh. Use traffic splitting to direct 10% of user traffic to the new version based on the user-agent header configured in the virtual service. **Most Voted**
-

Your team is running microservices in Google Kubernetes Engine (GKE). You want to detect consumption of an error budget to protect customers and define release policies. What should you do?

- A. Create SLIs from metrics. Enable Alert Policies if the services do not pass.
 - B. Use the metrics from Anthos Service Mesh to measure the health of the microservices.
 - C. Create a SLO. Create an Alert Policy on `select_slo_burn_rate`. **Most Voted**
 - D. Create a SLO and configure uptime checks for your services. Enable Alert Policies if the services do not pass.
-
-

Your organization wants to collect system logs that will be used to generate dashboards in Cloud Operations for their Google Cloud project. You need to configure all current and future Compute Engine instances to collect the system logs, and you must ensure that the Ops Agent remains up to date. What should you do?

- A. Use the `gcloud` CLI to install the Ops Agent on each VM listed in the Cloud Asset Inventory,
 - B. Select all VMs with an Agent status of Not detected on the Cloud Operations VMs dashboard. Then select Install agents.
 - C. Use the `gcloud` CLI to create an Agent Policy. **Most Voted**
 - D. Install the Ops Agent on the Compute Engine image by using a startup script
-
-

Your company has a Google Cloud resource hierarchy with folders for production, test, and development. Your cyber security team needs to review your company's Google Cloud security posture to accelerate security issue identification and resolution. You need to centralize the logs generated by Google Cloud services from all projects only inside your production folder to allow for alerting and near-real time analysis. What should you do?

- A. Enable the Workflows API and route all the logs to Cloud Logging.
- B. Create a central Cloud Monitoring workspace and attach all related projects.
- C. Create an aggregated log sink associated with the production folder that uses a Pub/Sub topic as the destination. **Most Voted**

- D. Create an aggregated log sink associated with the production folder that uses a Cloud Logging bucket as the destination.
-

You are configuring the frontend tier of an application deployed in Google Cloud. The frontend tier is hosted in nginx and deployed using a managed instance group with an Envoy-based external HTTP(S) load balancer in front. The application is deployed entirely within the europe-west2 region, and only serves users based in the United Kingdom. You need to choose the most cost-effective network tier and load balancing configuration. What should you use?

- A. Premium Tier with a global load balancer
 - B. Premium Tier with a regional load balancer
 - C. Standard Tier with a global load balancer
 - D. Standard Tier with a regional load balancer **Most Voted**
-

You recently deployed your application in Google Kubernetes Engine (GKE) and now need to release a new version of the application. You need the ability to instantly roll back to the previous version of the application in case there are issues with the new version. Which deployment model should you use?

- A. Perform a rolling deployment, and test your new application after the deployment is complete.
 - B. Perform A/B testing, and test your application periodically after the deployment is complete.
 - C. Perform a canary deployment, and test your new application periodically after the new version is deployed.
 - D. Perform a blue/green deployment, and test your new application after the deployment is complete. **Most Voted**
-

You are building and deploying a microservice on Cloud Run for your organization. Your service is used by many applications internally. You are deploying a new release, and you need to test the new version extensively in the staging and production environments. You must minimize user and developer impact. What should you do?

- A. Deploy the new version of the service to the staging environment. Split the traffic, and allow 1% of traffic through to the latest version. Test the latest version. If the test passes, gradually roll out the latest version to the staging and production environments.
 - B. Deploy the new version of the service to the staging environment. Split the traffic, and allow 50% of traffic through to the latest version. Test the latest version. If the test passes, send all traffic to the latest version. Repeat for the production environment.
 - C. Deploy the new version of the service to the staging environment with a new-release tag without serving traffic. Test the new-release version. If the test passes, gradually roll out this tagged version. Repeat for the production environment. **Most Voted**
 - D. Deploy a new environment with the green tag to use as the staging environment. Deploy the new version of the service to the green environment and test the new version. If the tests pass, send all traffic to the green environment and delete the existing staging environment. Repeat for the production environment.
-

You work for a global organization and run a service with an availability target of 99% with limited engineering resources.

For the current calendar month, you noticed that the service has 99.5% availability. You must ensure that your service meets the defined availability goals and can react to business changes, including the upcoming launch of new features.

You also need to reduce technical debt while minimizing operational costs. You want to follow Google-recommended practices. What should you do?

- A. Add N+1 redundancy to your service by adding additional compute resources to the service.
 - B. Identify, measure, and eliminate toil by automating repetitive tasks. **Most Voted**
 - C. Define an error budget for your service level availability and minimize the remaining error budget.
 - D. Allocate available engineers to the feature backlog while you ensure that the service remains within the availability target.
-

You are developing the deployment and testing strategies for your CI/CD pipeline in Google Cloud. You must be able to:

- Reduce the complexity of release deployments and minimize the duration of deployment rollbacks.
- Test real production traffic with a gradual increase in the number of affected users.

You want to select a deployment and testing strategy that meets your requirements. What should you do?

- A. Recreate deployment and canary testing
 - B. Blue/green deployment and canary testing **Most Voted**
 - C. Rolling update deployment and A/B testing
 - D. Rolling update deployment and shadow testing
-

You are creating a CI/CD pipeline to perform Terraform deployments of Google Cloud resources. Your CI/CD tooling is running in Google Kubernetes Engine (GKE) and uses an ephemeral Pod for each pipeline run. You must ensure that the pipelines that run in the Pods have the appropriate Identity and Access Management (IAM) permissions to perform the Terraform deployments. You want to follow Google-recommended practices for identity management. What should you do? (Choose two.)

- A. Create a new Kubernetes service account, and assign the service account to the Pods. Use Workload Identity to authenticate as the Google service account. **Most Voted**
 - B. Create a new JSON service account key for the Google service account, store the key as a Kubernetes secret, inject the key into the Pods, and set the `GOOGLE_APPLICATION_CREDENTIALS` environment variable.
 - C. Create a new Google service account, and assign the appropriate IAM permissions. **Most Voted**
 - D. Create a new JSON service account key for the Google service account, store the key in the secret management store for the CI/CD tool, and configure Terraform to use this key for authentication.
 - E. Assign the appropriate IAM permissions to the Google service account associated with the Compute Engine VM instances that run the Pods.
-

You are the on-call Site Reliability Engineer for a microservice that is deployed to a Google Kubernetes Engine (GKE) Autopilot cluster. Your company runs an online store that publishes order messages to Pub/Sub, and a microservice receives these messages and updates stock information in the warehousing system. A sales event caused an increase in orders, and the stock information is not being updated quickly enough. This is causing a large number of orders to be accepted for products that are out of stock. You check the metrics for the microservice and compare them to typical levels:

Microservice metrics	Typical state	Current state
Average CPU across all Pods	20% of Pod limit	30% of Pod limit
Average memory across all Pods	10% of Pod limit	10% of Pod limit
Pub/Sub subscription: Average oldest unacknowledged message age	347 milliseconds	8074 milliseconds
Pub/Sub subscription: Average undelivered messages	5 messages	14705 messages
Pub/Sub subscription: Average acknowledgment latency	312 milliseconds	354 milliseconds

You need to ensure that the warehouse system accurately reflects product inventory at the time orders are placed and minimize the impact on customers. What should you do?

- A. Decrease the acknowledgment deadline on the subscription.
 - B. Add a virtual queue to the online store that allows typical traffic levels.
 - C. Increase the number of Pod replicas. **Most Voted**
 - D. Increase the Pod CPU and memory limits.
-

Your team deploys applications to three Google Kubernetes Engine (GKE) environments: development, staging, and production. You use GitHub repositories as your source of truth. You need to ensure that the three environments are consistent. You want to follow Google-recommended practices to enforce and install network policies and a logging DaemonSet on all the GKE clusters in those environments. What should you do?

- A. Use Google Cloud Deploy to deploy the network policies and the DaemonSet. Use Cloud Monitoring to trigger an alert if the network policies and DaemonSet drift from your source in the repository.
- B. Use Google Cloud Deploy to deploy the DaemonSet and use Policy Controller to configure the network policies. Use Cloud Monitoring to detect drifts from the source in the repository and Cloud Functions to correct the drifts.

- C. Use Cloud Build to render and deploy the network policies and the DaemonSet. Set up Config Sync to sync the configurations for the three environments.
 - D. Use Cloud Build to render and deploy the network policies and the DaemonSet. Set up a Policy Controller to enforce the configurations for the three environments. **Most Voted**
-
-

You are using Terraform to manage infrastructure as code within a CI/CD pipeline. You notice that multiple copies of the entire infrastructure stack exist in your Google Cloud project, and a new copy is created each time a change to the existing infrastructure is made. You need to optimize your cloud spend by ensuring that only a single instance of your infrastructure stack exists at a time. You want to follow Google-recommended practices. What should you do?

- A. Create a new pipeline to delete old infrastructure stacks when they are no longer needed.
 - B. Confirm that the pipeline is storing and retrieving the terraform.tfstate file from Cloud Storage with the Terraform gcs backend. **Most Voted**
 - C. Verify that the pipeline is storing and retrieving the terraform.tfstate file from a source control.
 - D. Update the pipeline to remove any existing infrastructure before you apply the latest configuration.
-
-

You are creating Cloud Logging sinks to export log entries from Cloud Logging to BigQuery for future analysis. Your organization has a Google Cloud folder named Dev that contains development projects and a folder named Prod that contains production projects. Log entries for development projects must be exported to dev_dataset, and log entries for production projects must be exported to prod_dataset. You need to minimize the number of log sinks created, and you want to ensure that the log sinks apply to future projects. What should you do?

- A. Create a single aggregated log sink at the organization level.
 - B. Create a log sink in each project.
 - C. Create two aggregated log sinks at the organization level, and filter by project ID.
 - D. Create an aggregated log sink in the Dev and Prod folders. **Most Voted**
-
-

Your company runs services by using multiple globally distributed Google Kubernetes Engine (GKE) clusters. Your operations team has set up workload monitoring that uses Prometheus-based tooling for metrics, alerts, and generating dashboards. This setup does not provide a method to view metrics globally across all clusters. You need to implement a scalable solution to support global Prometheus querying and minimize management overhead. What should you do?

- A. Configure Prometheus cross-service federation for centralized data access.
 - B. Configure workload metrics within Cloud Operations for GKE.
 - C. Configure Prometheus hierarchical federation for centralized data access.
 - D. Configure Google Cloud Managed Service for Prometheus. **Most Voted**
-

You need to build a CI/CD pipeline for a containerized application in Google Cloud. Your development team uses a central Git repository for trunk-based development. You want to run all your tests in the pipeline for any new versions of the application to improve the quality. What should you do?

- A. 1. Install a Git hook to require developers to run unit tests before pushing the code to a central repository.
2. Trigger Cloud Build to build the application container. Deploy the application container to a testing environment, and run integration tests.
3. If the integration tests are successful, deploy the application container to your production environment, and run acceptance tests.
- B. 1. Install a Git hook to require developers to run unit tests before pushing the code to a central repository. If all tests are successful, build a container.
2. Trigger Cloud Build to deploy the application container to a testing environment, and run integration tests and acceptance tests.
3. If all tests are successful, tag the code as production ready. Trigger Cloud Build to build and deploy the application container to the production environment.
- C. 1. Trigger Cloud Build to build the application container, and run unit tests with the container.
2. If unit tests are successful, deploy the application container to a testing environment, and run integration tests.
3. If the integration tests are successful, the pipeline deploys the application container to the production environment. After that, run acceptance tests.
- D. 1. Trigger Cloud Build to run unit tests when the code is pushed. If all unit tests are successful, build and push the application container to a central registry.
2. Trigger Cloud Build to deploy the container to a testing environment, and run integration tests and acceptance tests.

3. If all tests are successful, the pipeline deploys the application to the production environment and runs smoke tests **Most Voted**

The new version of your containerized application has been tested and is ready to be deployed to production on Google Kubernetes Engine (GKE). You could not fully load-test the new version in your pre-production environment, and you need to ensure that the application does not have performance problems after deployment. Your deployment must be automated. What should you do?

- A. Deploy the application through a continuous delivery pipeline by using canary deployments. Use Cloud Monitoring to look for performance issues, and ramp up traffic as supported by the metrics. **Most Voted**
 - B. Deploy the application through a continuous delivery pipeline by using blue/green deployments. Migrate traffic to the new version of the application and use Cloud Monitoring to look for performance issues.
 - C. Deploy the application by using kubectl and use Config Connector to slowly ramp up traffic between versions. Use Cloud Monitoring to look for performance issues.
 - D. Deploy the application by using kubectl and set the spec.updateStrategy.type field to RollingUpdate. Use Cloud Monitoring to look for performance issues, and run the kubectl rollback command if there are any issues.
-